Writeup for MLSys 2026 Graph Scheduling Competition

Ryang Sohn

May 4, 2026

1 Introduction and Approach

This writeup outlines the overall approach we have taken to implement a computation graph optimizer.

The overall approach was inspired by two prior works: Korch[1] and Welder[2]. From Korch, we have adopted a kernel selection strategy using BLP (Binary Linear Programming) to allow selecting schedules that recompute some operations. Also, we have implemented a selection algorithm to determine retained tensors based on the `GraphConnecting` algorithm introduced in Welder.

The overall flow of the optimizer is as follows:

1. The optimizer enumerates convex subgraphs of the whole graph.
2. Convex subgraphs are benchmarked using the performance model.
3. Using the costs obtained from the benchmark, the optimizer selects the subgraphs using BLP.
4. The selected subgraphs are further optimized by selecting tensors to retain.

1.1 Convex Subgraph Enumeration

Korch defines the notion of "convex subgraph" as follows: a subgraph $G' = (V', E')$ of a DAG $G = (V, E)$ is *convex* if for every directed path in G whose endpoints both lie in V', every intermediate vertex on that path is also in V'. This guarantees that the subgraph can be scheduled as a single fused kernel without splitting an operation away from any of its transitive dependencies that lie "inside" the kernel boundary.

We adopt this definition and use convex subgraphs as units of kernel selection in the next step, subgraph selection. The problem constraints dictate that all subgraphs must be connected, so we make sure that all enumerated convex subgraphs are connected (i.e., each subgraph has only one component).

Rather than the pairwise-difference enumeration described by Korch, we use a seed-and-grow procedure: for each operation $v \in V$, we initialize $S = \{v\}$ and recursively try to add each frontier neighbor w (a predecessor or successor of some node in S). For convexity, we maintain the ancestor and descendant bitsets $\mathrm{anc}(S)$ and $\mathrm{desc}(S)$ precomputed once via topological sweeps; a candidate $S \cup \{w\}$ is convex iff $\mathrm{anc}(S \cup \{w\}) \cap \mathrm{desc}(S \cup \{w\}) \subseteq S \cup \{w\}$. Visited subsets are deduplicated by a hash of their bitset, and growth halts at a fixed maximum fusion size, since fusion benefit plateaus once the working set exceeds fast memory.

1.2 Benchmarking Convex Subgraphs

For each enumerated subgraph, the optimizer searches `(w, h, k)` tile sizes in parallel and picks the one minimizing input and output memory traffic, preferring larger tiles to break ties.

After choosing tile size, the performance model determines the cost of the subgraph using the rules defined in the problem statement, also in parallel.

1.3 Subgraph Selection

The benchmarked subgraphs are then fed into the kernel selector. The kernel selector uses the BLP formulation introduced in Korch[1] to determine a set of optimal kernels to execute. Constraints are added to the BLP problem instance to ensure that all output tensors are computed and dependencies are not violated.

In this implementation, we used the `good_lp` crate to interface with the HiGHS LP solver, which is suitable because it is fast and allows static linking into the binary. To ensure that the solver finishes under time constraints, we capped the BLP solver's running time to 10 seconds.

1.4 Retention and Traversal

The selected kernels are then optimized further by retaining some of the tensors in fast memory. In this problem, the retained tensors of a subgraph can only be consumed by the next executed subgraph. By leveraging this, we can view the problem of choosing which tensors to retain as a problem of how to partition a subgraph into multiple subgraphs, where each partitioned subgraph's outputs are retained in the current subgraph and consumed by the next subgraph. We implemented a memory-level selection algorithm inspired by Welder's `GraphConnecting` algorithm[2]. In this implementation, fast memory is treated as "higher level" than ephemeral memory, and we added beam search to consider more candidates than a greedy algorithm.

For traversal order, our solver always uses the "snake" traversal order to maximize implicit data reuse. In the setting of this competition, this is optimal because implicit data reuse is only possible across adjacent steps of subgraph execution.

2 Results

We evaluated the optimizer on the five public benchmarks, comparing the total latency it produces against a naive baseline that schedules each operation as its own subgraph at the largest feasible tile. Latencies are reported in the cost model's native units; the speedup column is the naive latency divided by the optimized latency.

Benchmark	Ops	Naive latency	Optimized latency	Speedup
`mlsys-1`	5	576,716.80	576,716.80	1.00×
`mlsys-5`	19	1,459,268.27	1,160,329.60	1.26×
`mlsys-9`	32	80,530,636.80	76,373,155.84	1.05×
`mlsys-13`	63	22,165,913.60	22,031,564.80	1.01×
`mlsys-17`	103	5,081,850.88	4,983,971.84	1.02×

The optimizer never regresses against the baseline. The largest gain is on `mlsys-5`, whose graph has enough fusible pointwise neighborhoods to amortize matmul output writes; on `mlsys-1`, all five operations are already memory-bound at their native tile, so no fusion improves on the per-op schedule.

References

[1] Muyan Hu, Ashwin Venkatram, Shreyashri Biswas, Balamurugan Marimuthu, Bohan Hou, Gabriele Oliaro, Haojie Wang, Liyan Zheng, Xupeng Miao, Jidong Zhai, and Zhihao Jia. Optimal kernel orchestration for tensor programs with korch. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, page 755769, New York, NY, USA, 2024. Association for Computing Machinery.

[2] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. Welder: Scheduling deep learning memory access via tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 701–718, Boston, MA, July 2023. USENIX Association.